

# DeepPersona 코드 학습교재

모듈 06 — 실행 · 재현 · 스크립트 (마지막)

---

학습 정리 덱 (Claude와 함께)

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338)

**한눈에** 앞 5개 모듈로 이해한 파이프라인을 실제로 돌려 재현한다 — 환경→pk1→smoke→sweep.

## 얻는 것

- `.venv` + `.env` 환경 설정
- 깨진 `pk1` 재생성(진짜 스크립트)
- `smoke test`로 동작 확인
- `sweep`로 README "1MB" 검증
- 재현 체크리스트(런북)

▶ 튜토리얼은 입문용 고수준, 본 교재는 코드 밀착 심화 — 상보적으로 함께 보면 좋다.

## 참고 자료(입문 튜토리얼)

- ValleyJin/ `Tutorial-Builder`  
`output/Deeppersona/index.md`
- 5장 구성 ↔ 본 교재 모듈 대응(다음 장)
- `references.bib` 에 등재

0. 튜토리얼 ↔ 본 교재 지도
1. 환경 설정 (.venv + .env)
2. pickle 재생성
3. 동작 확인 (smoke & sweep)
4. 재현 체크리스트
5. 교재 마무리

## 0. 튜토리얼 $\leftrightarrow$ 본 교재 지도

---

**쉽게 말하면** 같은 코드를 두 시선으로: 튜토리얼(비유 입문) + 본 교재(코드 심화). 대응표로 오가며 학습.

| 튜토리얼 장         | 본 교재 모듈              |
|----------------|----------------------|
| 1. .env 시크릿 로딩 | 06 §1 (환경 설정)        |
| 2. 인물 생성 엔진    | 03 (시드) + 05 (값 채우기) |
| 3. 계층적 속성 변환   | 02 (속성 트리 Stage 1)   |
| 4. 속성 선택기      | 04 (임베딩 + 5:3:2)     |
| 5. 피클 파일 재생성   | 06 §2 (pkl 재생성)      |

**한눈에** 튜토리얼=직관("왜/무엇"), 본 교재=재현("어떻게/코드"). 겹쳐 보면 이해가 단단해진다.

## 1. 환경 설정 (.venv + .env)

---

## 환경 — Python 3.11 `venv` + 패키지

```
python3.11 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install openai sentence-transformers scikit-learn \
                    numpy tqdm geonamescache python-dotenv
```

- `.venv` : 프로젝트 전용 격리 환경(시스템 Python 오염 방지). `.gitignore` 에 포함.
- **핵심 패키지**: `openai` (LLM·임베딩), `sentence-transformers` (트리 dedup, 모듈 02), `scikit-learn` (코사인), `geonamescache` (위치, 모듈 03), `python-dotenv` (.env).

▶ 설치본: `openai 2.40`, `sentence-transformers 5.5`(`torch 2.12`), `numpy 2.4`. (study.md §1)

```
from dotenv import load_dotenv
load_dotenv() # .env 를환경변수로로드
GEONAMES_USERNAME = os.getenv("GEONAMES_USERNAME", "demo")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY: # 없으면즉시실패실수 ( 방지)
    raise RuntimeError("OPENAI_API_KEY is not set. Add it to .env ...")
client = OpenAI(api_key=OPENAI_API_KEY)
```

- **왜:** API 키를 코드에 "sk-..." 로 박으면 유출 위험. `.env` (gitignore)로 분리.
- `load_dotenv()` : `.env` 의 `KEY=VALUE` 를 `os.environ` 에 주입 → 어디서든 `os.getenv` 로 조회.
- **조기 실패:** 키 없으면 `raise` — 빈 키로 조용히 돌아 실패하는 것 방지.
- **원본 문제:** 5개 파일에 `OPENAI_API_KEY="OPENAI_API_KEY"` placeholder 하드코딩 → 이 패턴으로 교체(study.md §2).

▶ `.env` (실제 시크릿, gitignore) + `.env.example` (템플릿, 커밋)로 분리.

## 2. pickle 재생성

---

## 깨진 pickle 재생성 — 복구 아닌 재계산

```
def flatten_paths(node, prefix=""):      # 트리 -> leaf 경로목록선택기와 (동일)
    if isinstance(node, dict) and not node: return [prefix]      # 빈{} = leaf
    ...                                  # 재귀로 X.Y.Z 경로수집
paths = flatten_paths(taxonomy)          # 개 2,297
for i in range(0, len(paths), BATCH_SIZE): # 개씩 256 배치
    resp = client.embeddings.create(model="text-embedding-ada-002", input=batch)
    all_embeddings.extend([d.embedding for d in resp.data])
embeddings = np.asarray(all_embeddings, dtype=np.float64)
assert embeddings.shape == (len(paths), 1536) # (2297, 1536) 검증
pickle.dump({"attribute_paths": paths, "embeddings": embeddings}, f)
```

- **원리**: ada-002 weights는 고정 → 같은 입력 = 같은 벡터. "복구"가 아니라 **재계산**(study.md §7).
- **입력 보존**: 슬롯 경로는 `large_attributes.json` 에 그대로 → 재료가 남아 있어 재생성 가능.
- **배치**: 256개씩 묶어 API 호출 수↓. `np.float64`, shape `(2297,1536)` `assert` 로 무결성 확인.
- **안전**: 깨진 원본을 `.broken` 으로 백업 후 덮어씀.

▶ 비용 ≈\$0.002, 1~3분. 튜토리얼 5장은 같은 개념을 비유로 설명(코드는 예시적).

**쉽게 말하면** 원본 repo의.pkl이 정확히 6.8MB에서 잘려 있었다(정상 27MB). 산수로 원래 정체를 역추론했다.

- 증상: `pickle data was truncated`
- `pickletools` : numpy 배열 직렬화 중 절단, 기대 28,225,536 bytes
- $28,225,536 \div 8(\text{float64}) \div 1,536 = 2,297$
- $2,297 = \text{leaf 수}$ ,  $1,536 = \text{ada-002 차원}$  → 정체 확정

### 재생성 결과

27.05 MB 정상.pkl. 이후 vector search 정상 동작(빈 프로필 → 실제 속성).

**한눈에** 원본 repo(upstream 포함) 자체의 결함 — 전송 오류 아님. `regenerate_embeddings.py` 로 해결.

▶ 상세 forensic: study.md §6. 재현자는 이.pkl 무결성을 가장 먼저 확인해야 한다.

### 3. 동작 확인 (smoke & sweep)

---

# smoke test — end-to-end 1회

```
sys.path.insert(0, str(REPO_ROOT / "generate_user_profile"))
from generate_profile import generate_single_profile
profile = generate_single_profile(template=None, profile_index=0, attribute_count=100)
if not profile: sys.exit("FAILED: empty profile")
out_path.write_text(json.dumps(profile, ensure_ascii=False, indent=2))
print(f"size: {out_path.stat().st_size:,} bytes")
```

- **목적:** 시드→선택→값채우기→Summary 전 과정이 도는지 **한 번에** 확인.
- **신호:** 정상이면 수 KB JSON. **1.9KB**면 pkl 문제(속성 0개) → §2로.
- **실행:** `.venv/bin/python scripts/test_single_profile.py` (repo root에서).

▶ 재생성 전엔 1.9KB(0 attrs), 재생성 후 count=100에서 9.0KB(53 attrs)로 회복.

## sweep — README "1MB" 주장 검증

**쉽게 말하면** 속성 수를 늘려가며 프로파일 크기를 측정해, 광고된 "페르소나당 1MB"가 실제로 나오는지 확인한다.

파일: `scripts/test_attribute_count_sweep.py`

| count | 시간    | 크기      | attrs |
|-------|-------|---------|-------|
| 100   | 31.7s | 9.0 KB  | 53    |
| 200   | 38.0s | 13.3 KB | 94    |
| 300   | 55.7s | 20.1 KB | 145   |
| 350   | 74.2s | 27.4 KB | 200   |

### 결론

- 최대 27.4 KB — 1MB의 2.7%(약 37배 격차)
- 선형(0.14 KB/attr): 1MB엔 ~7,300 attrs 필요(트리 max는 2,297)
- 병목 = 모듈 05의 카테고리 drop 버그(6/19 처리)

**한눈에** README의 "1MB narrative"는 현재 코드 경로로는 도달 불가 — 매핑 수정이 선결 (study.md §13).

## 4. 재현 체크리스트

---

**쉽게 말하면** 새 환경에서 이 순서대로만 하면 페르소나가 재현된다.

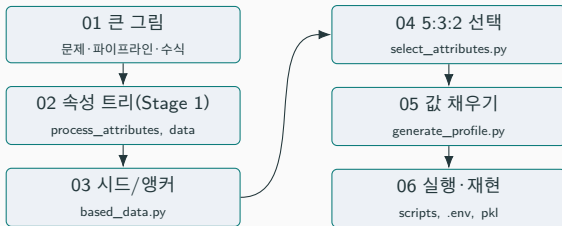
```
1 # 1) 환경
2 python3.11 -m venv .venv
3 .venv/bin/pip install openai sentence-transformers scikit-learn \
4                      numpy tqdm geonamescache python-dotenv
5 # 2) 시크릿
6 cp .env.example .env          # 편집: OPENAI_API_KEY=sk-... 필요시 ( GEONAMES_USERNAME )
7 # 3) 임베딩캐시재생성필수 (! 원본은 pkl 손상)
8 .venv/bin/python scripts/regenerate_embeddings.py      # -> 27MB pkl
9 # 4) 동작확인
10 .venv/bin/python scripts/test_single_profile.py       # -> 수 KB JSON
11 # 5) 선택() 스케일검증
12 .venv/bin/python scripts/test_attribute_count_sweep.py
```

- **3번이 관문:** pkl 재생성을 건너뛰면 4번이 1.9KB 빈 프로필로 실패.
- **get\_completion 견고성:** LLM 호출은 `max_retries=3` 재시도 내장(일시 오류 방어).

▶ 출력은 `output/` 에 저장(gitignore). API 비용이 드니 `count` · 횟수를 작게 시작.

## 5. 교재 마무리

---



**한눈에** Stage 1(트리) → Stage 2(시드→선택→채우기) → 실행·재현. 이제 코드를 읽고 재현할 수 있다.

## 파이프라인

- 트리 1회 구축(추출→필터→병합→검증)
- 시드: 인구통계(테이블)+성격·서사(LLM)
- 선택: ada-002 임베딩 + 5:3:2
- 채우기: 카테고리 순차·조건부(depth-first)

## 알아들 결합/함정

- 원본 `pk1` 손상 → 반드시 재생성
- top-level 19 vs 12(중복 variant)
- 카테고리 6/19만 처리 → 27KB
- 값 100자 캡 → 얇은 narrative

**한눈에** "1MB"까지 가려면: 카테고리 매핑 수정 + variant 정규화 + narrative 길이↑ — 모두 필요.

▶ 더 깊은 배경·전략(한국어 페르소나, 고대인 NPC 등)은 루트 `study.md` §10-15.

## 교재 완주 ☒

6개 모듈 끝 — 이제 코드를 이해하고 재현할 수 있다